Group Members:

Ujjwal - Q1
Rishi - Q2
Praveer - Q3

Question 1

Used the unicodedata module to normalize the input using the “NFKC”, and then to remove the I explicitly removed them and added whitespace to punctuation marks (again identified by unicodedata library) and returning the string

The createcorpus function first converts the text into a frequency table of unique words.

For each distinct word the first character is kept the same , every subsequent character is prefixed with and the word is stored once, along with its frequency in the corpus

The corpus stores it as (tokens, frequency)

Getcharcounts takes in the corpus and it simply iterates through each of the tokens and increases the count of each token by simply adding to the frequency, it returns dictionary that represents the total number of times each symbol appears across the entire corpus

Getpairwisecounts works the same way as Getcharcounts but it counts occurrence of a pair

Getpairscore takes in both of the getcharcounts and getpairwisecounts dictionary and evaluates the score for each of the unique pairs, by the formula occ(a,b)/occ(a)*occ(b) then simply returns that new dictionary

Gethighestliklihood takes a dictionary and sorts it lexicography and returns the first ie pair with the highest score lexicographically

The merge function takes a list of corpus and a pair it then if it finds the pair in the old corpus it combines it, if the pair is like s,##a it makes it sa, if pair is ##s,##a it makes it ##sa otherwise if its another word it simply adds it to the new corpus and then returns the new corpus, The function preserves the frequency of each word and returns a new corpus in which all occurrences of the selected pair have been replaced by the merged token

The Train wordpiece simply uses the above functions all in a loop till the desired vocab limit and produces the corpus , this is the function that fits our init vocmap which is vocabulary mapping and vocmap_inv which is the inverse of the dictionary of vocmap then it returns the sorted vocabulary

The encode function iterates through the provided text and it greedily checks so it starts from the end of the word, if it finds a similar token in our vocab is looks up the index in the vocmap and replaces it, unless it goes through the whole word and still hasn't found something in our vocmap it will return <UNK> ie 0 and then it prepends 1 and postpends 2 to the list
Simply returns the list of token Id

The decode function iterates through the list of tokens and simply matches those who start with ## to out vocmap_inv and if it finds a known id it replaces it with its text version.

The rest of the functions are simply helper functions to open files, save to files and run the above functions.

Deliverables:

Tokenizer→ wordpiece.pynb
Provided dataset → hinditextdata.txt
Sample tester → sample.txt

Rest of the names are straightforward

Question 2

Note

The notebook realFastText.ipynb contains the whole implementation, including classes, ngram generation, model training, and the experiments. I saved the model parameters from there.

The python file q2.py is the one which is to be run in the demo. It has the class definitions, and it loads the model directly using the saved params from the file saved_params.pt in the data folder.

Dimensions of model = 40

Classes

1. **WordpieceTokenizer:** The same tokenizer made in Q1.
2. **NgramGenerator:** Implements char n-gram extraction from a dataset and manages its vocabulary (ngram2id, id2ngram). Used to (i) build the subword dictionary and (ii) compute subword IDs (ie., word embeddings, which are the sum of ngrams) for any word (including OOV)
3. **Trainer:** Implements Skip-Gram using PyTorch. The trainable parameters (embedding matrices) are Z and V, corresponding to embeddings for ngrams (subwords) and embeddings for output/context words respectively.
    a. Also implements negative sampling, subsampling, skip-gram pair generation, training loop, inference (word vectors), nearest neighbors, and important n-gram" analysis.
4. **FastText:** Acts sort of as a wrapper class - it wraps the above three together. I mainly built it to make the application look object-oriented.

Note: Most functions have **docstrings** that explain their purpose and inputs/outputs. The report below highlights the essential ones.

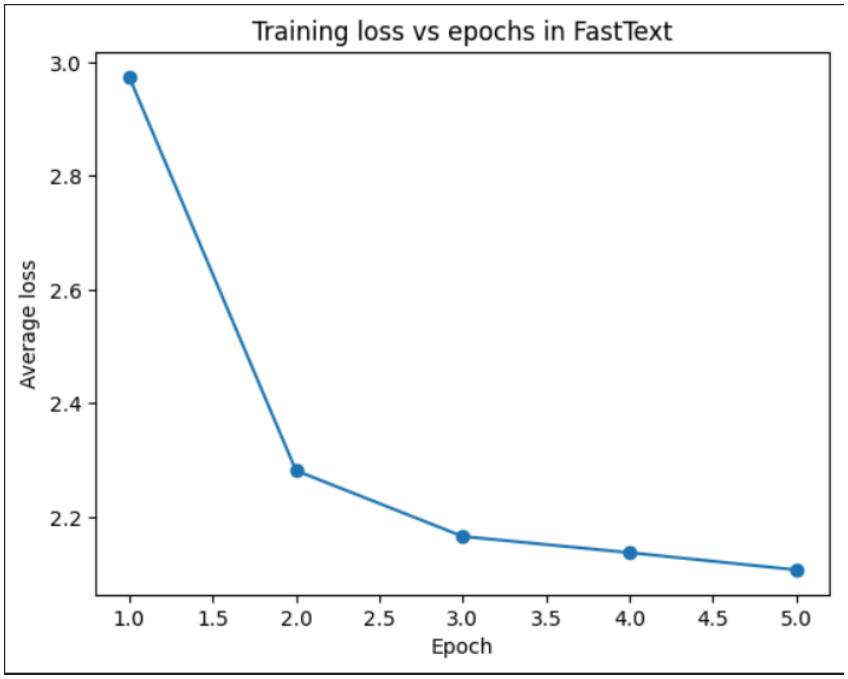
Essential Methods

1. **extract_ngrams:** Adds word boundaries, then extracts all ngrams of lengths 3 to 5, and the word itself.
2. **ngram_vocab:** Extracts ngrams for each word in a preprocessed file using the above method, then builds ngram vocabulary (ngram2id, id2ngram)
3. **word_to_ngram_ids:** extracts ids of all ngrams associated with a word, from the built ngram vocab.
4. **write_bags_file:** using normal file reading and writing methods, writes word bags to a file ngram_bags.txt, using the method above.

5. **word_vocab:** builds word vocabulary from word counts. Special tokens like UNK are excluded.
6. build_negative_sampler and sample_negatives: handle negative sampling for skipgram.
7. build_subsampling: builds subsampling using the formula for probability of dropping which was done in class
8. **generate_skipgram_pairs:** generates (center_word, context_word) pairs on which the model is later trained
9. **sgns_step:** it basically represents one weight update of the skipgram model. Implemented using standard pytorch methods.
10. **train:** trains the sgns (skipgram negative sampling) model.
11. **nearest_neighbors:** finds nearest neighbors over vocab words, which are represented by their fasttext representations. The relation/distance between two words is their cosine similarity
12. **save and load:** save and load the model params (like Z, V, ngram2id, etc).
13. **oov_demo:** uses nearest_neighbors to demonstrate how OOV words are represented.
14. **important_ngrams:** assigns a score to each ngram of a word. Uses cosine similarity for this.

Training Loss vs Epochs

The avg loss started out high, but reduced significantly over the first few steps (see screenshot below graph). Even though the loss would sometimes shoot up a bit at a specific step, the margin was very small, and it went down during the next step. After about three and a half epochs, the decrease in loss was very low, so a better result couldn't be achieved.



```
[epoch 1] step=10,000 avg_loss=3.8894
[epoch 1] step=20,000 avg_loss=2.9492
[epoch 1] step=30,000 avg_loss=2.5978
[epoch 1] step=40,000 avg_loss=2.4607
[epoch 1] epoch_avg_loss=2.9743
[epoch 2] step=10,000 avg_loss=2.4142
[epoch 2] step=20,000 avg_loss=2.2722
[epoch 2] step=30,000 avg_loss=2.2362
[epoch 2] step=40,000 avg_loss=2.2042
[epoch 2] epoch_avg_loss=2.2817
[epoch 3] step=10,000 avg_loss=2.2404
[epoch 3] step=20,000 avg_loss=2.1913
[epoch 3] step=30,000 avg_loss=2.1068
[epoch 3] step=40,000 avg_loss=2.1245
[epoch 3] epoch_avg_loss=2.1657
[epoch 4] step=10,000 avg_loss=2.1865
[epoch 4] step=20,000 avg_loss=2.1488
[epoch 4] step=30,000 avg_loss=2.1074
[epoch 4] step=40,000 avg_loss=2.1061
[epoch 4] epoch_avg_loss=2.1372
[epoch 5] step=10,000 avg_loss=2.1899
[epoch 5] step=20,000 avg_loss=2.1318
[epoch 5] step=30,000 avg_loss=2.0734
[epoch 5] step=40,000 avg_loss=2.0326
[epoch 5] epoch_avg_loss=2.1069
```

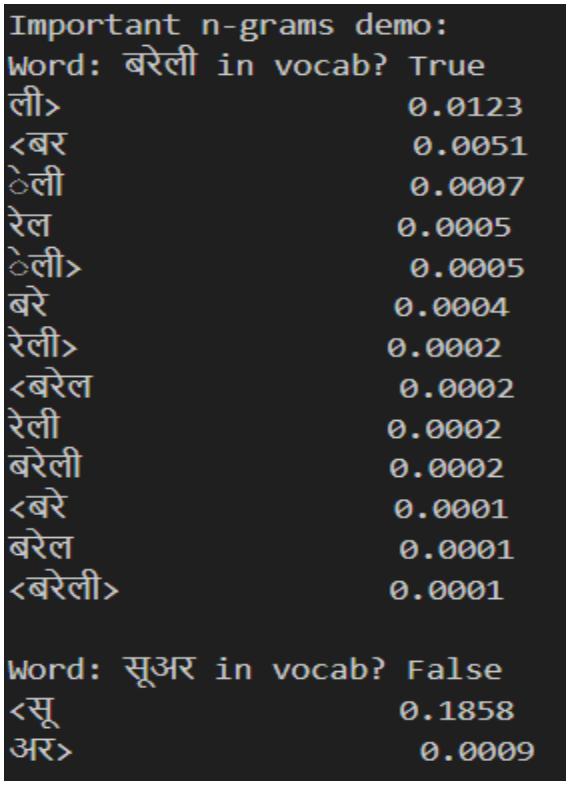
Experiments & Analysis

1. **OOV Words:** I used दिल्लीवालापन as an Out of vocab word. The model was able to represent it using already known subwords. It also found related known words using nearest neighbors.

```
OOV word vector demo:
OOV word: दिल्लीवालापन
In vocab? False
#known ngrams used: 29
vector shape: (40,)
vector norm: 3.75500750541687
vector (first 8): [-0.6169242262840271,
दिल्लीवालो                    0.9842
सेवन                           0.9766
ध्यान                         0.9757
यापन                           0.9753
मंज़ूर                        0.9753
ज्ञानदास                      0.9741
दिल                            0.9729
घंटे                          0.9726
तौल                            0.9725
आनंदी                         0.9723
```

2. **Important n-grams as morphemes**

   I tried to find important ngrams for both in-vocab and out of vocab words. The results were not surprising (since I knew what would happen), but exciting nonetheless. The smallest ngram units which had meanings (morphemes) were rated higher than others. Also even for OOV words, we were able to get ngrams which constitute them.



Question 3

Model Architecture

The model is a feed-forward architecture comprising of:

- A word embedding lookup table
- Concatenation of embeddings for a fixed-size context window
- One (two (ablation)) fully connected hidden layers with non-linear activation (tanh)
- A linear output projection over the vocabulary

Two architectural variants were evaluated:

- Single hidden layer
- Two hidden layers

Two Context size variants were evaluated for both the architectural variants:

- Context size three
- Context size five

Hyperparameters

The following hyperparameters were used consistently across all experiments:

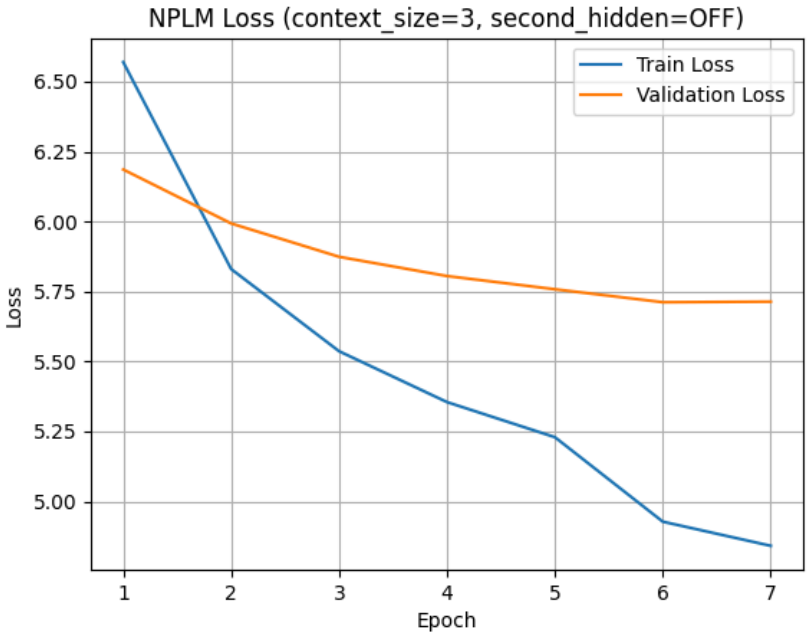
- Context sizes: 3, 5
- Embedding dimension: 128
- Hidden layer size: 512 (used for both hidden layers when the second hidden layer is enabled)
- Batch size: 128
- Epochs: 7 (however, all four configurations achieved their best validation loss at epoch 6. Therefore, results are reported for models selected at epoch 6, and training was limited to 7 epochs in the final runs)
- Optimizer: Adam
- Learning rate: 0.001
- Weight decay: 2e-4
- Gradient clipping norm: 1.0
- UNK replacement probability (training): 0.01
- Learning rate scheduler: StepLR
- Step size: 5
- Gamma: 0.5
- Loss function: CrossEntropyLoss

- Corpus: wmt-news-crawl-hi.txt
- Vocabulary: Provided vocab_q3.txt

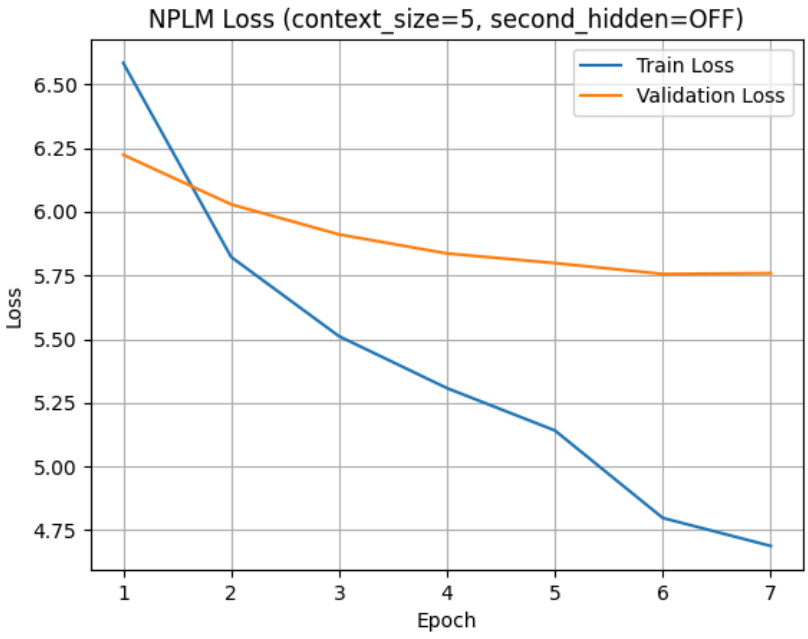
Training and Validation Loss Curves

(note early stopping was implemented and all 4 models had best val at epoch 6 so thats what was saved)

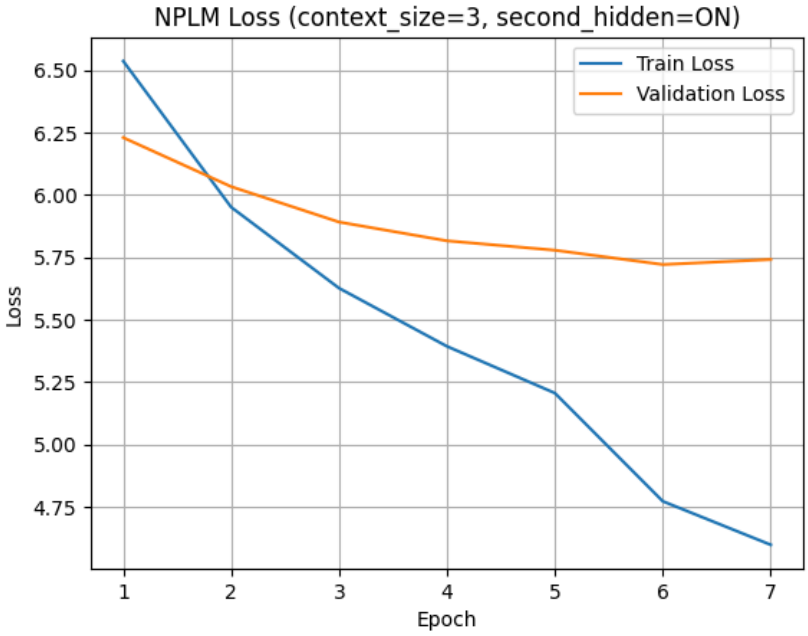
Model 1: Context size 3 only 1 Hidden Layer



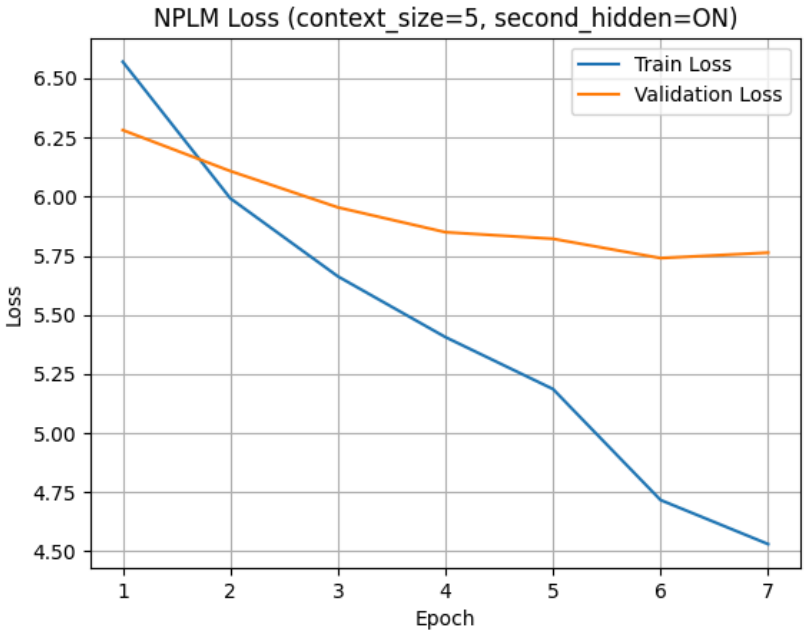
Model 2: Context size 5 only 1 Hidden Layer



Model 3: Context size 3 and 2 Hidden Layers



Model 5: Context size 5 and 2 Hidden Layers



Evaluation Matrix

Model 1: Context size 3 only 1 Hidden Layer

Dataset	**Perplexity**	**Accuracy (%)**
Training	97.6402	25.96
Validation	302.6189	19.67

Model 2: Context size 5 only 1 Hidden Layer

Dataset	**Perplexity**	**Accuracy (%)**
Training	81.9550	26.84
Validation	316.5025	19.53

Model 3: Context size 3 and 2 Hidden Layers

Dataset	Perplexity	Accuracy (%)
Training	72.7672	28.13
Validation	305.3726	19.71

Model 5: Context size 5 and 2 Hidden Layers

Dataset	Perplexity	Accuracy (%)
Training	67.0886	28.40
Validation	311.3570	19.64

Observation:

Increasing model capacity (larger context size and additional hidden layer) improved training performance, as seen by decreasing training perplexity and increasing training accuracy.
However, validation perplexity and accuracy remain largely unchanged (slightly worsened) across models, indicating limited generalization to unseen data.
This suggests that while larger context windows and deeper architectures help the model fit the training data better, they do not significantly improve predictive performance on validation data, especially in a simplistic model

Files:

- WPtokenizer.py: WordPiece tokenizer implementation reused directly from Q1.
- main.py: Entry-point script providing a menu-based interface to train, evaluate, and generate text using the models.
- nplm_dataset.py: Dataset construction code that preprocesses text, tokenizes input, and generates sliding context target pairs for training and evaluation.
- nplm_model.py: Implementation of the Neural Probabilistic Language Model (NPLM) architecture.

- nplm_trainer.py: Training loop for all experimental configurations, including model saving, loss tracking, and learning-rate scheduling.
- eval.py: Evaluation script to compute perplexity and next-token prediction accuracy on training, validation, or test-style datasets.
- nplm_inference.py: Text generation module that produces continuations given seed sentences using trained models.
- view_metadata.py: Utility script to inspect saved model metadata and training statistics.
- loss_metadata_saved.txt: Output file generated by view_metadata.py, containing recorded hyperparameters, metrics, and loss summaries.
- ./plot: training-validation plots for all 4 models
- ./losses: saved losses of all 4 models
- ./model: saved model+metadat for all 4 models
- ./genrated_out: output for nplm_inference file

Assumptions:

- During evaluation, the user may provide a corpus different from the training data. Therefore, the system allows specifying whether the input is a test corpus, if marked as test, perplexity and next-token accuracy are computed on the entire file without a train/validation split, otherwise the default train-validation split is used.
- Early stoping is allowed
- Set seed is allowed

References:

https://www.unicode.org/ucd/
https://spacy.io/api/tokenizer
Piotr Bojanowski et al. “Enriching Word Vectors with Subword Information”. In: Transactions of the Association for Computational Linguistics 5 (2017). Ed. by Lillian Lee, Mark Johnson, and Kristina Toutanova, pp. 135–146. doi: 10.1162/tacl_a_00051. url: https://aclanthology.org/Q17-1010/
Tomas Mikolov et al. “Distributed Representations of Words and Phrases and their Compositionality”. In: Advances in Neural Information Processing Systems. Ed. by C.J. Burges et al. Vol. 26. Curran Associates, Inc., 2013. url: https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf
Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. “A Neural Probabilistic Language Model”. In: Advances in Neural Information Processing Systems. Ed. by T. Leen, T. Dietterich, and V. Tresp. Vol. 13. MIT Press, 2000. url: https://proceedings.neurips.cc/paper_files/paper/2000/file/728f206c2a01bf572b5940d7d9a8fa4c-Paper.pdf